

EE147 Assignment #2 Questions

How many thread blocks are used to perform reduction with an input size of 100,000 and thread block size of 512 threads?

- 1) Since each thread is inputting 2 elements, we have $2 \times 512 \text{ threads} = 1024 \text{ elements}$.
 $100000 / 1024$ is 97.656 or about 98 blocks.

How much shared memory (in Bytes) is used by each thread block?

- 2) Given our code `__shared__ float partialSum[2 * BLOCK_SIZE]`, and our block size being 512 leaves us with 1024 floats. With a float being 4 bytes, $1024 \text{ floats} \times 4 \text{ bytes}$ is 4096 bytes. 4096 bytes of shared memory is used by each thread block.

Recall we do not perform a full reduction in our assignment, but rather have each thread block reduce a partial sum that will be sent back to the host. How many partial sums will be transferred back to the host?

- 3) The number of partial sums is equivalent to the number of blocks. Since we calculated 98 blocks being used, we have 98 partial sums.

For the naive reduction kernel, how many steps execute without divergence? How many steps execute with divergence?

- 4) Since the block size is 512, $\log_2(512) = 9$ steps + the first step (stride of 1). Using the segment:

```
for (unsigned int stride = 1; stride <= blockDim.x; stride *= 2)
{
    __syncthreads();

    if (t % stride == 0)
        partialSum[2 * t] += partialSum[2 * t + stride];

    countWarpDistribution();
}
```

We can see some threads will be inactive because not every thread condition will be true. Therefore, only 1 step will be without divergence while all other 9 steps will have divergence. Not every thread will be active, which is the cause of warp divergence.

For the optimized reduction kernel, how many steps execute without divergence?
How many steps execute with divergence?

- 5) We still have 10 steps, however in optimized reduction, we are using a different code segment. This code segment goes as follows:

```
for (unsigned int stride = blockDim.x; stride > 0; stride /= 2)
{
    if (t < stride)
    {
        partialSum[t] += partialSum[t + stride];
    }

    countWarpDistribution();
    __syncthreads();
}
```

This code segment starts with the greatest step, then works downwards. For the first 5 steps, there will be no warp divergence. However, once we hit 16, the first half of the warp will be active and the second half will be inactive. Therefore, stride 16, 8, 4, 2, and 1 all experience warp divergence.

From the results of your kernel implementations, which kernel performed better? Use profiling statistics (such as kernel execution time, the warp occupancy distribution, etc.) to support your claim.

- 6) Viewing our kernel results, the optimized reduction performs better because of fewer divergent warps and a better execution time. Less warp divergence means better utilization of thread. Optimized reduction also has a better execution time by a significant margin, citing it to be the better option.

How does the warp occupancy distribution compare between the two Reduction implementations?

- 7) The warp occupancy distribution for the naive reduction method leaves threads scattered and does not properly utilize all the threads within a warp. The optimized version utilizes all the threads within a warp and leaves each warp either fully active or diverging. The

distribution is much more clean and stacked together compared to the naive reduction method.

Explain why GPUs suffer from warp divergence.

- 8) GPUs suffer from warp divergence because warps are 32 threads executed together. However, not every command can be perfectly fit into the 32 threads. The if statements can break up the computing process. The GPU will execute the true step first, then execute the false step. However, since both processes can not run at the same time, the idle threads take up resources and also increase computation time. If the entire warp was all active, then the execution would be 1 step and resources would be fully utilized.